Practice midterm 2 (with solutions)

TC1037 Algorithmic Thinking · Group 510 · Tecnológico de Monterrey, Campus Querétaro

Practice — download, solve on paper in 80 minutes, then check the solutions

Name: _____ **Matrícula:** _____

80 minutes · 100 points · individual · on paper. No computer, phone, notes or calculator. Write in pen or dark pencil; code must be legible and indented. **Part I** (30 pts): write *exactly* what Python prints — every line, in order. **Part II** (70 pts): write the program. Suggested time: 15 minutes for Part I, 65 for Part II. **This is a practice paper: same format, same difficulty, different problems.**

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	5	5	5	5	5	5	23	23	3	20	1	100
Score:												

Part I — Read and predict

(30 points)

1. **Arithmetic operators.** What does Python print? [5]

```
print(10 - 6 / 3 * 2 + 15 // 4)
```

Solution: `9.0`

Why

`10 - 6 / 3 * 2 + 15 // 4`
= `10 - 2.0 * 2 + 15 // 4` — `/` first (leftmost of `* / // %`) — and `/` always gives a float
= `10 - 4.0 + 15 // 4` — `*`
= `10 - 4.0 + 3` — `//`: 4 fits in 15 three whole times
= `6.0 + 3` = `9.0` — `+ -` left to right; float plus int is a float

Not `9` (one `/` makes the result a float) and not `3.0` (that is reading left to right, ignoring precedence).

2. **Boolean operators.** What does Python print? [5]

```
print(4 > 3 and not 2 == 2.0 or 7 != 7)
```

Solution: `False`

Why

`4 > 3 and not 2 == 2.0 or 7 != 7`
= `True and not True or False` — all comparisons first; `2 == 2.0` is True (same value)
= `True and False or False` — `not` flips only the value right after it
= `False or False` — `and`: True only if both are True
= `False` — `or` last

3. **Conditional structure.** What does Python print? [5]

```
x = 9
y = 9
if x > y:
    print("A")
elif x == y and x % 3 == 0:
    print("B", x * 2)
elif x == y:
    print("C")
else:
    print("D")
print("end")
```

Solution: `B 18`
`end`

Why

`x > y`: 9 > 9 is False → skip.

`x == y and x % 3 == 0`: True **and** (`9 % 3` = 0, so True) → True → prints `B` and `9 * 2` = 18.

The next `elif x == y` is also True, but only the *first* True branch runs. Then `end` (outside the `if`). Output: `B 18` / `end`.

4. **Functions.** What does Python print? [5]

```
def k(n):
    n = n - 1
    return n * 2

def main():
    n = 5
    m = k(n)
    m = k(m)
    print(n, m)

main()
```

Solution: `5 14`

Why

First call `k(5)`: k's own `n` becomes 4, returns `4 * 2` = 8 → main's `m = 8`.

Second call `k(m)` = `k(8)`: k's `n` becomes 7, returns 14 → `m = 14`.

`n = n - 1` changes *k's* `n`, not main's: main's `n` is still 5. Output: `5 14`.

5. **The while loop.** What does Python print? [5]

```
i = 0
while i < 10:
    i = i + 3
    if i % 2 == 0:
        print(i)
print("last", i)
```

Solution: `6`
`12`
`last 12`

Why

test `i < 10`?	`i` after `i = i + 3`	even?	prints
yes (0)	3	no	—
yes (3)	6	yes	`6`
yes (6)	9	no	—
yes (9)	12	yes	`12`
no (12)			

The test is checked only at the top of each turn, so `i` can pass 10 *inside* the turn. After the loop, `print("last", i)` prints `last 12`.

6. **The for loop.** What does Python print? [5]

```
total = 0
for i in range(5, 0, -2):
    total = total + i
    print(total)
```

Solution:
```
5
8
9
```

Why

`range(5, 0, -2)`: start at 5, jump by -2, stop *before* 0 → `i` takes 5, 3, 1.

`total`: $0+5=5$, $5+3=8$, $8+1=9$. The `print` is inside the loop → three lines: `5`, `8`, `9`.

Part II — Write the program **(70 points)**

Write **one** Python program with the following parts. All `input()` and all validation happen in `main()`; the two functions **return** their result and never print or read. Write each function's purpose in a one-line comment above it, and under each loop a one-line comment saying why it stops (`while`) or how many times it runs (`for`).

7. The function `first_power_above(base, limit)` receives two integers $base \geq 2$ and $limit \geq 1$ and **returns** the smallest exponent e such that $base^e > limit$. Use a `while` loop; do not use `math.log`. [23]

   Examples: `first_power_above(2, 10)` → `4` `first_power_above(2, 16)` → `5` `first_power_above(10, 999)` → `3` `first_power_above(3, 1)` → `1`

8. The function `sum_squares(n)` receives an integer $n \geq 1$ and **returns** $1^2 + 2^2 + \cdots + n^2$. Use a `for` loop. [23]

   Examples: `sum_squares(1)` → `1` `sum_squares(3)` → `14` `sum_squares(4)` → `30` `sum_squares(10)` → `385`

9. The function `menu()` that displays: `1. first_power_above 2. sum_squares 3. Exit` [3]

10. The function `main()` that, inside a `while` loop: shows the menu, reads the option, and [20]
    - option 1: reads base (2 or more) and limit (1 or more); if any value is out of range displays a clear message, otherwise calls `first_power_above` and displays the result with a label;
    - option 2: reads n (must be 1 or more); same rule, calling `sum_squares`;
    - option 3: displays `Goodbye.` and the program ends;
    - any other option: displays `Invalid option.` and the program ends.

    Use `if`/`elif`/`else` to choose the option.

11. The last line of the script calls `main()`. [1]

Technical requirements — 2 points off for each one missing: meaningful variable names in Python style · a clear prompt for every input · readable, labelled output.

Reference solution

```python
# Reference solution — Practice midterm 2 (with solutions)

def first_power_above(base, limit):
    exponent = 0
    power = 1
    while power <= limit:
        power = power * base
        exponent = exponent + 1
    return exponent


def sum_squares(n):
    total = 0
    for i in range(1, n + 1):
        total = total + i ** 2
    return total
```

```python
def menu():
    print("1. first_power_above")
    print("2. sum_squares")
    print("3. Exit")


def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            base = int(input("Base (2 or more): "))
            limit = int(input("Limit (1 or more): "))
            if base < 2 or limit < 1:
                print("Invalid input: base must be 2 or more and limit 1 or more.")
            else:
                print("Smallest exponent:", first_power_above(base, limit))
        elif option == 2:
            n = int(input("n (1 or more): "))
            if n < 1:
                print("Invalid input: n must be 1 or more.")
            else:
                print("Sum of the squares:", sum_squares(n))
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:
            print("Invalid option.")
            keep_going = False


main()
```

How to think about `first_power_above`

Keep `power` $=$ `base`$^{\texttt{exponent}}$, starting at $1 = base^0$. While `power` is still `<=` `limit`, multiply by `base` once more and count it. It stops at the *first* power greater than `limit` — `power` grows every turn.

Check the edges: `(3, 1)`: $1 \leq 1 \rightarrow$ one turn $\rightarrow$ `power` 3, `exponent` 1 $\rightarrow$ returns 1. Writing `<` instead of `<=` breaks `(2, 16)`: it stops at 16 and returns 4 instead of 5.

How to think about `sum_squares`

Accumulator from 0; `range(1, n + 1)` so that `n` itself is included; add `i ** 2` each turn.

Check: `sum_squares(3)` $= 1 + 4 + 9 = 14$. With `range(1, n)` you get 5 — the last term is missing.

How to think about `main`

A flag `keep_going` controls the `while`. Every option follows the same four steps: **read** (`int(input(…))`) $\rightarrow$ **validate** (`if` out of range $\rightarrow$ message) $\rightarrow$ **call** the function $\rightarrow$ **print with a label**.

Option 3 and any invalid option set the flag to False, so the loop ends. Validation is done in `main`, never inside the function: the function only computes and `returns`.

How Part II is graded (the academia's rubric)

Each part of the program (the two functions, `menu`, `main`, the call) is scored on this scale, as a percentage of its points:

Excelente 100%	**Muy bien 85%**	**Regular 70%**	**Deficiente 50%**	**Muy deficiente 20%**	**Sin evidencia 0%**
No syntax errors; works correctly, including the boundary cases in the examples.	Correct approach; minimal logic or syntax slips.	At most two syntax errors and/or small logic errors; the approach is closely related to what was asked.	Approach loosely related, or related but with more than two syntax errors.	Code unrelated to the problem.	No code.

Part I: 5 points per question, all or nothing — every line of the output, in order. Technical requirements: -2 each, from Part II.